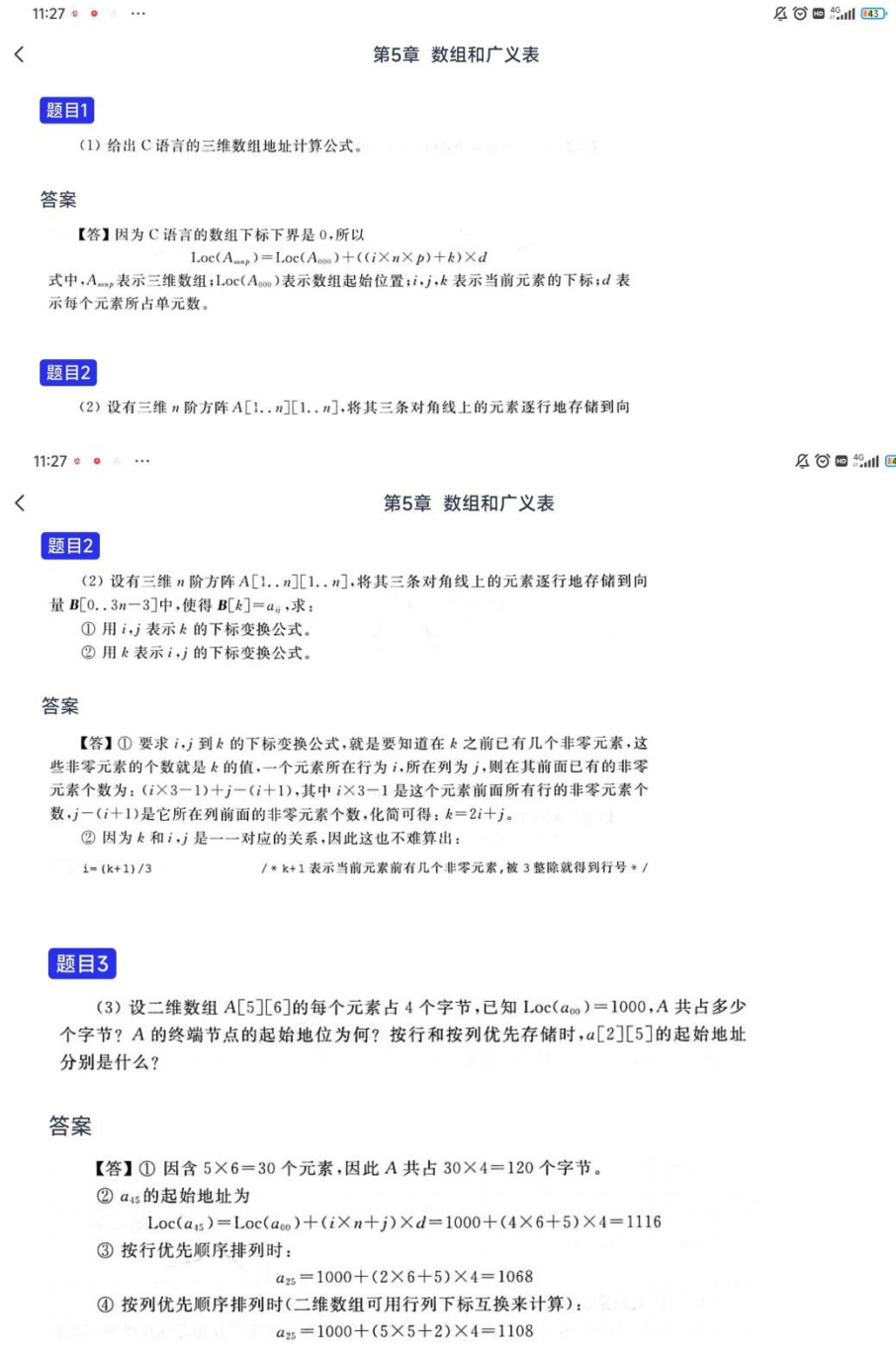
以下是我的模糊记忆 >-<
以下只是考过的点，而不是具体题目，展现的是示例样题 0w0
1. 复杂度大小排序，类似这样

（1）将下列复杂度由小到大重新排序：2^n、$n!$、n^2、10000、$\log_2 n$、$n\times\log_2 n$。

答案

【答】$10000<\log_2 n<n\times\log_2 n<n^2<2^n<n!$。

2. 计算数组地址，类似这样（好像期末考了，期中不记得有没有，截了 3 张不是因为它是重点，而是因为它的题型多变~）



11:27 ··· 4G 43

第5章 数组和广义表

题目1

（1）给出 C 语言的三维数组地址计算公式。

答案

【答】因为 C 语言的数组下标下界是 0，所以

$$Loc(A_{mnp})=Loc(A_{000})+((i\times n\times p)+k)\times d$$

式中，A_{mnp} 表示三维数组；$Loc(A_{000})$ 表示数组起始位置；i,j,k 表示当前元素的下标；d 表示每个元素所占单元数。

题目2

（2）设有三维 n 阶方阵 $A[1..n][1..n]$，将其三条对角线上的元素逐行地存储到向

11:27 ··· 4G 43

第5章 数组和广义表

题目2

（2）设有三维 n 阶方阵 $A[1..n][1..n]$，将其三条对角线上的元素逐行地存储到向量 $B[0..3n-3]$ 中，使得 $B[k]=a_{ij}$，求：

① 用 i,j 表示 k 的下标变换公式。

② 用 k 表示 i,j 的下标变换公式。

答案

【答】① 要求 i,j 到 k 的下标变换公式，就是要知道在 k 之前已有几个非零元素，这些非零元素的个数就是 k 的值，一个元素所在行为 i，所在列为 j，则在其前面已有的非零元素个数为：$(i\times 3-1)+j-(i+1)$，其中 $i\times 3-1$ 是这个元素前面所有行的非零元素个数，$j-(i+1)$ 是它所在列前面的非零元素个数，化简可得：$k=2i+j$。

② 因为 k 和 i,j 是一一对应的关系，因此这也不难算出：

```
i=(k+1)/3                    /* k+1 表示当前元素前有几个非零元素，被 3 整除就得到行号 */
```

题目3

（3）设二维数组 $A[5][6]$ 的每个元素占 4 个字节，已知 $Loc(a_{00})=1000$，A 共占多少个字节？A 的终端节点的起始地位为何？按行和按列优先存储时，$a[2][5]$ 的起始地址分别是什么？

答案

【答】① 因含 $5\times 6=30$ 个元素，因此 A 共占 $30\times 4=120$ 个字节。

② a_{45} 的起始地址为

$$Loc(a_{45})=Loc(a_{00})+(i\times n+j)\times d=1000+(4\times 6+5)\times 4=1116$$

③ 按行优先顺序排列时：

$$a_{25}=1000+(2\times 6+5)\times 4=1068$$

④ 按列优先顺序排列时（二维数组可用行列下标互换来计算）：

$$a_{25}=1000+(5\times 5+2)\times 4=1108$$

3. 广义表的 head，tail，写出结果或根据结果写 head-tail 表达式，这应该可以说肯定会考~

题目7

(7) 设广义表 $L=((),())$，则 head(L)，tail(L)是f

答案

【答】head(L)=()。
tail(L)=(())。
L 的长度为 2。
L 的深度为 2。

题目8

(8) 广义表的 head 和 tail 运算，把原子 d 分别从下
$L_1=(((((a),b),d),e))$；$L_2=(a,($
中分离出来。

答案

【答】head(tail(head(head(L_1))))。
head(head(head(tail(head(tail(L_2))))))。

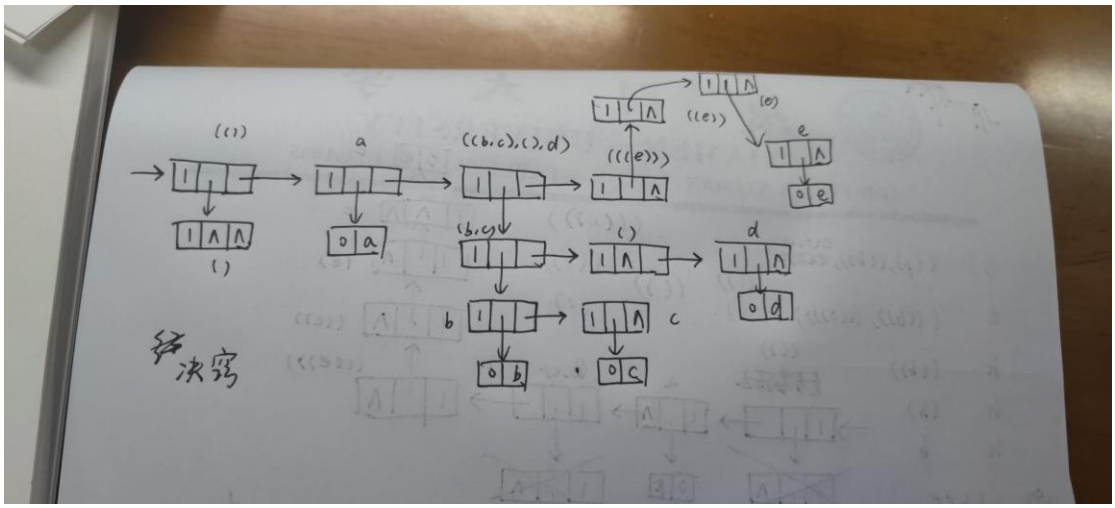
题目9

(9) 下列广义表运算的结果：
① head(tail(((a,b),(c,d),(e,f))))。
② tail(head(((a,b),(c,d),(e,f))))。
③ head(tail(head(((a,b),(e,f)))))。
④ tail(head(tail(((a,b),(e,f)))))。
⑤ tail(tail(head(((a,b),(e,f)))))。

答案

【答】①(c,d)；②(b)；③b；④(f)；⑤()。

4. 画广义表，大概长这样，没找到样题，还有什么深度、宽度之类的



5. KMP（必考题）：它的匹配过程模拟、写出 next 数组、nextval 数组都要掌握
6. 二叉树前中后遍历，两种遍历结果画出确定的树等

题目8

(3) 某二叉树节点的先根遍历序列为 $EACBDGF$，
二叉树节点的后根遍历序列为(　　)。
A. $BDCAFGE$　　B. $BDCFAGE$　　C. 1

题目17

(2) 若二叉树中各节点的值均不相同，则由二叉树的先根遍历序列和中根遍历序列，或由其中根遍历序列和后根遍历序列均能唯一地确定一棵二叉树，但由先根遍历序列和后根遍历序列却不一定能唯一地确定一棵二叉树。

① 已知一棵二叉树的先根遍历序列和中根遍历序列分别为 *ABDGHCEFI* 和 *GDHBAECIF*，请画出此二叉树。

② 已知一棵二叉树的中根遍历序列和后根遍历序列分别为 *BDCEAFHG* 和 *DECBHGFA*，请画出此二叉树。

③ 已知一棵二叉树的先根遍历序列和后根遍历序列分别为 *AB* 和 *BA*，请画出这两棵不同的二叉树。

答案

【答】

7. 画线索二叉树（两种结点格式），我记得当时练了很多这个，但有没有考我忘记了 orz
8. 树和二叉树的转换

题目22

(7) 画出图 6-6 中的树转换后得到的二叉树。

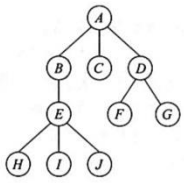


图 6-6

答案

【答】见图 6-7。



9. 森林和二叉树的转换

题目23

(8) 将图 6-8 所示的森林转换为二叉树。

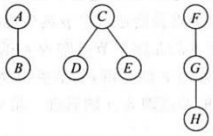
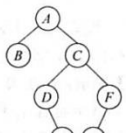


图 6-8

答案

【答】见图 6-9。



10. 最后一题考了手写代码，具体题目我忘记了，但我记得掌握好递归和非递归方法进行中序遍历非常有利于写出这道题

可以参考中序遍历的过程说明自行模拟出入栈示意图。先序遍历的非递归算法如下：

```
void PreOrder2(BiTree T){
InitStack(S); BiTree p=T;          //初始化栈 S；p 是遍历指针
    while(p||!IsEmpty(S)){         //栈不空或 p 不空时循环
        if(p){                     //一路向左
            visit(p);Push(S,p);    //访问当前结点，并入栈
            p=p->lchild;           //左孩子不空，一直向左走
        }
        else{                      //出栈，并转向出栈结点的右子树
            Pop(S,p);              //栈顶元素出栈
            p=p->rchild;           //向右子树走，p 赋值为当前结点的右孩子
        }                          //返回 while 循环继续进入 if-else 语句
    }
}
```

根据分析可以写出中序遍历的非递归算法如下：

```
void InOrder2(BiTree T){
InitStack(S); BiTree p=T;          //初始化栈 S；p 是遍历指针
    while(p||!IsEmpty(S)){         //栈不空或 p 不空时循环
        if(p){                     //一路向左
            Push(S,p);             //当前结点入栈
            p=p->lchild;           //左孩子不空，一直向左走
        }
        else{                      //出栈，并转向出栈结点的右子树
            Pop(S,p);visit(p);     //栈顶元素出栈，访问出栈结点
            p=p->rchild;           //向右子树走，p 赋值为当前结点的右孩子
        }                          //返回 while 循环继续进入 if-else 语句
    }
}
```

算法思想：后序非递归遍历二叉树是先访问左子树，再访问右子树，最后访问根结点。结合

```
void PostOrder(BiTree T){
    InitStack(S);
    p=T;
    r=NULL;
    while(p||!IsEmpty(S)){
        if(p){                                  //走到最左边
            push(S,p);
            p=p->lchild;
        }
        else{                                   //向右
            GetTop(S,p);                        //读栈顶结点（非出栈）
            if(p->rchild&&p->rchild!=r)//若右子树存在，且未被访问过
                p=p->rchild;                    //转向右
            else{                               //否则，弹出结点并访问
                pop(S,p);                       //将结点弹出
                visit(p->data);                 //访问该结点
                r=p;                            //记录最近访问过的结点
                p=NULL;                         //结点访问完后，重置 p 指针
            }
        }//else
    }//while
}
```

最后，对于期中笔试，做书本作业题应该会很有帮助（期末会灵活一点，和期中不同）
哈夫曼树倒是去年一直没有考过，不知道今年是会继续不考还是反而会去考 QAQ